

Event Management Platform

Event Management Platform

Objective

The main goal of the MVP is to create a simple and functional event management platform that allows event organizers to create and promote events, while attendees can browse and register for those events.

Core Features

Feature 1

1. Event Discovery, Details, Creation, and Promotion (4 point)
 - **Landing Page:** Display a list of upcoming events.
 - **Event Browsing:** Customers can browse events, filter by category/location, and view event details.
 - **Search Bar:** Implement a search bar with debounce functionality.
 - **Responsiveness** is a must.
 - **Event Creation:** Event organizers can create events with details such as name, price, start date, end date, available seats, description, ticket types (if applicable), etc.
 - **Pricing:** Events can be free or paid. If paid, customers are charged accordingly.
 - **Promotions:** Event organizers can create limited-time voucher promotions specific to events, with defined start and end dates.
2. Event Transaction (4 point)
 - **Purchasing:** Customers can create transactions to buy event tickets.
 - **Transaction Statuses:** There are six statuses: waiting for payment, waiting for admin confirmation, done, rejected, expired and canceled.
 - **Payment Proof:** After choosing a ticket and checking out, a 2-hour countdown is shown for uploading the payment proof.
 - **Automatic Status Changes:** Transactions expire if no payment proof is uploaded within 2 hours. If the organizer doesn't accept/reject within 3 days, the transaction is automatically canceled.
 - **Rollbacks and Seat Restoration:** Points, vouchers, or coupons used in transactions are returned if the transaction is canceled or expired. Additionally, **available seats are restored.**

- **Point Usage:** Customers can use points to reduce payment amounts.(ex: event tickets price is IDR 300.000 while your points balance is 20.000, you could use it and get IDR 280.000 as the final price)
 - Uses only **IDR** in each prices of items
3. Event Reviews and Ratings (2 point)
- **Reviews:** Customers can leave reviews and ratings only after attending the event.
 - **Organizer Profile:** Show ratings and reviews on the event organizer's profile.

Feature 2

1. User Authentication and Authorization (2 point)
- **Account Creation:** Customers must create an account to attend events.
 - **Roles:** There are two roles: customer and event organizer.
 - **Referral Registration:** Customers can register using a referral number.
 - **Referral Generation:** Referral numbers are generated for new users and cannot be changed.
 - **Role-Based Access:** Protect pages based on user roles.
2. Referral System, Profile, and Prizes (4 point)
- **Referral Rewards:** Users registering with a referral get a discount coupon, and the referrer gets 10,000 points.
 - **Points Expiration:** Points expire 3 months after being credited. (ex: today is 28 Dec 2023 and there are 3 people using your referral number, your balance would be 30.000 and available until 28 March 2024.
 - **Coupon Expiration:** Discount coupons after registering with referral are valid for 3 months.
 - **Profile:** Customers and Event organizers can edit their profiles, including updating their profile picture, changing their password, and resetting their password if forgotten.
3. Event Management Dashboard (4 point)
- **Dashboard Access:** Organizers can view and manage their events (ex: edit events, etc.), transactions, and basic statistics.
 - **Statistics Visualization:** Display event data in graphical visualizations and reports by year, month, and day.
 - **Transaction Management:** Organizers can accept, reject, and view user payment proofs.
 - **Notification Emails:** Customers receive email notifications when their transaction is accepted or rejected. Ensure points/vouchers/coupons are **returned** if used in rejected transactions. Additionally, **available seats are restored**.

- **Attendee List:** Show the list of attendees for each event, including name, ticket quantity, and total price paid.

Clues:

1. **Voucher Discount:** This is provided by the event organizer and can only be used for specific events organized by them.
2. **Reward / Coupon Discount:** This is provided by the application system and can be used for all events.

Notes:

1. Protected route should implemented
2. Responsiveness is a must
3. Implement debounce on search bar
4. Implement popup dialog as confirmation on modify data
5. Create unit test on each flows
6. Handle if there are no items shows in filter or search
7. Implement SQL transaction on modify action that more than one action
8. Provide data that relevant to the projects

References

<https://www.eventbrite.com/>

<https://www.eventbookings.com/explore-events/>

<https://www.tickettailor.com/discover/>

<https://www.viagogo.com/ww>

<https://www.loket.com/>

NEWEST MINPRO REQUIREMENT

MINI PROJECT

Requirements & Guidelines

Objective

Build a full stack web application that solves a real-world problem. Students are free to choose their application domain. The assessment focus is not on the business domain chosen, but on:

- Implementation quality & technical architecture
- Scalability & maintainability
- Code quality & best practices
- User experience & design consistency

Example Application Domains

E-Commerce • Booking System • Educational Platform • Finance Tracker • Inventory Management • Social Platform • Healthcare App • Food Ordering • Project Management • and more

Tech Stack Requirements

All projects must use the following tech stack. Any technology outside this list requires prior approval from the lecturer.

Layer	Technology
Frontend	Next.js, Typescript
Backend	Node.js with Express, Typescript
Database	PostgreSQL
Version Control	Git + GitHub
ORM	Prisma ORM

Core Technical Requirements

1.1 Authentication & Authorization

The application must implement authentication and role-based authorization.

Minimum Requirements:

- New account registration
- Login with email & password
- Logout and token invalidation
- Password hashing
- Protected routes / pages
- Role-based access control (RBAC)

Constraints

- Minimum 2 user roles (e.g. admin & user, seller & buyer, teacher & student)
- Implement JWT — token stored in httpOnly cookie or localStorage
- Refresh token mechanism (optional, bonus points)
- Sensitive routes must be protected on the backend side (middleware)
- Frontend must block page access that does not match the user role

1.2 CRUD & Data Management

The application must have structured data management with clear relationships between entities.

Minimum Requirements:

- Minimum 7 main entities / resources (database tables)
- Full CRUD for at least 3 main entities
- Proper relational database design
- Soft delete implementation on main entities

Constraints

- Minimum 1 one-to-many relationship
- Minimum 1 many-to-many relationship (with junction table)
- Data validation is mandatory on the backend
- Implement soft delete
- Error responses must be consistent (use a uniform JSON format)
- Include seed data for demo / testing purposes

1.3 Search, Filter & Pagination

The application must support proper data exploration features for an optimal user experience.

Minimum Requirements:

- Search with debounce
- Filtering by at least 2 parameters
- Sorting (ascending / descending)
- Pagination (page-based or cursor-based)

Constraints

- Debounce minimum 300ms before triggering an API call
- All search, filter, sort, and pagination logic must be processed on the backend (not the frontend)
- Queries must use efficient WHERE, ORDER BY, LIMIT, OFFSET clauses
- Empty states must be handled with an informative UI
- Pagination response must include: data, total, page, totalPages, limit

1.4 Transactional Flow

The application must implement at least one complex transactional / business flow.

Example Transactional Flows (adapt to your domain):

- Place order → payment → update stock → send notification email
- Course enrollment → capacity check → generate certificate → send email
- Schedule booking → availability check → confirmation → send reminder

Constraints

- Must use SQL transactions when modifying multiple related tables
- Data consistency must be maintained — if one step fails, the entire operation must be rolled back
- Edge cases must be handled (out of stock, slot full, payment failure, etc.)
- Status flow must be clearly defined and cannot be arbitrarily skipped

1.5 Dashboard & Data Visualization

The application must provide a dashboard page with statistical visualizations relevant to the chosen domain.

Minimum Requirements:

- Minimum 3 summary cards/statistics
- Minimum 2 chart/graph visualizations
- Statistics filter by time range/date

Constraints

- Must use a chart library (Recharts, Chart.js, or ApexCharts)
- Dashboard data must come from backend aggregation queries (COUNT, SUM, AVG, GROUP BY)
- Dashboard must only be accessible by specific roles (e.g. admin)
- Loading state and error state on charts must be handled properly

1.6 File Upload

The application must support a file upload feature using cloud storage.

Minimum Requirements:

- Minimum 1 file upload feature (e.g. profile photo, product thumbnail, document)

Constraints

- File validation must be performed on the backend: file type (MIME type) and maximum size
- Files must be stored using cloud — not on the local server
- Use Multer as Express middleware to handle upload processing
- The resulting file URL must be stored in the database
- Old files must be deleted from cloud storage when a user updates their file

1.7 Mailer System

The application must implement an automated email sending mechanism at relevant points in the user flow.

Minimum Requirements:

- At least 1 automated email flow

Constraints

- Use Nodemailer library
- Email templates must use proper HTML (not plain text)
- Email sending must be done asynchronously (must not block the API response)
- Example email triggers: successful registration, transaction confirmation, password reset, schedule reminder
- Email credentials must be stored in environment variables (.env) — never hardcoded

Code Quality & Best Practices

2.1 Error Handling

Standards

- Global error handler middleware in Express (app.use at the very end)
- All error responses must use a consistent JSON format: { success, message, errors }
- Correct HTTP status codes must be used (200, 201, 400, 401, 403, 404, 409, 500)
- Frontend must handle API errors and display user-friendly messages
- No console.log statements left in the production build
- Try-catch must be used on all async operations on the backend

2.2 Environment & Security

Standards

- All credentials (DB password, JWT secret, API keys) must be stored in .env
- .env file must be added to .gitignore — must not be committed to GitHub
- Include a .env.example file as a template for reviewers
- CORS must be configured correctly (only allow necessary origins)
- JWT secret must use a strong, unique string
- No hardcoded credentials or URLs in the code

Deployment Requirements

All applications must be deployed to the cloud and publicly accessible before the demo session.

3.1 Deployment Checklist

What Must Be Deployed

- Frontend → deploy to Vercel (connect directly from the GitHub repository)
- Backend / API → deploy to Vercel / Railway
- Database → PostgreSQL on Railway or Supabase
- Environment variables must be configured on each platform (not in the code)

3.1 Post Deployment Requirements

Must Be Fulfilled Before Demo

- The production application must run without errors (no crashes during demo)
- The production database must be migrated and seeded with sufficient dummy data

- All features to be demonstrated must be tested in the production environment (not localhost)
- Frontend URL and backend API URL must be included in the README.md
- Deployment URLs must be submitted together with the GitHub repository link

Github Repository Requirements

The GitHub repository is part of the assessment for professionalism and project documentation.

4.1 README.md

The README.md must contain the following information:

- Project name and brief description
- Main application features (bullet points)
- Tech stack used
- ERD (Entity Relationship Diagram) of the database
- Steps to run the project locally (setup guide)
- Deployment URLs (frontend & backend)
- Demo accounts (email & password for each role)

4.2 Git Workflow

Commit & Branch Standards

- Minimum 20 meaningful commits throughout the development period
- Commit messages must use a descriptive format (e.g. feat: add pagination to product list)
- Use branching: main for production, develop for development
- Direct push to the main branch is not allowed
- .gitignore must exclude: node_modules, .env, dist, build

Scoring Rubric & Assessment Guide

Mini Project

Overview

This rubric applies to all mini project submissions regardless of the chosen application domain. Each student in a 2-person group is assessed individually. The assessment focuses on technical implementation quality, not on the business domain chosen.

Scores are assigned on a 6-point scale:

- 100 (Excellent)
- 90 (Very Good)
- 80 (Good)
- 70 (Above Average)
- 60 (Average)
- 40 (Poor)

A score of 0 is recorded for non-submission or non-attendance.

Assessment Criteria & Weights

Criterion	Weight	What's Evaluated
Frontend Evaluation	20%	Next.js UI/UX, responsiveness, state management, routing
Backend Evaluation	20%	Express API, Prisma ORM, auth, error handling, security
Core Features	20%	All 7 technical requirements: auth, CRUD, search/filter/pagination, transaction, dashboard, file upload, mailer
Feature Testing	10%	App stability during demo; bug severity; unit tests (bonus)
Industry Readiness	15%	Code quality, deployment, Git workflow, README, security
Code Understanding	15%	Individual Q&A — ability to explain own code and decisions
TOTAL	100%	

Score Scale Reference

Score	Label	Description
100	Excellent	Exceptional — exceeds expectations across the board
90	Very Good	Meets all goals with only minor issues
80	Good	Meets most goals; minor delays or quality issues
70	Above Average	Meets minimum requirements; room for improvement
60	Average	Partially meets requirements; several issues or gaps
40	Poor	Significant issues; missed requirements; low quality
0	Not Submitted	Did not participate or submit the project

Grading Notes

1. Final Score = (Examiner 1 Score × 55%) + (Examiner 2 Score × 45%) for Technical Skills
2. Soft Skills (Presentation & Communication) are scored separately and averaged between examiners.
3. Each student in a 2-person group is assessed individually — scores may differ within the same group.

1. Frontend Evaluation

Weight: 20% | Evaluates the quality and completeness of the Next.js frontend implementation including UI/UX design, responsiveness, routing, state management, form validation, and overall user experience.

Score	Label	Descriptor
100	Excellent	Exceptional UI/UX with pixel-perfect responsive design. Fully functional across all device sizes. Advanced features like skeleton loading, optimistic UI, and error boundaries implemented. Consistent design system throughout.
90	Very Good	Clean, responsive UI with good design consistency. All pages accessible and navigable. Proper loading states, error messages, and empty states handled. Minor visual inconsistencies only.
80	Good	Functional UI meeting most requirements. Responsive on major breakpoints. Form validation and error handling present but not exhaustive. A few UX flow issues or minor visual gaps.
70	Above Average	Core pages built and usable. Basic responsiveness implemented. Some error handling missing. Design is functional but lacks polish. A few features partially implemented or with noticeable bugs.
60	Average	Partially meets requirements. Several pages missing or broken. Limited responsiveness. Minimal error handling. UI is inconsistent or visually incomplete across sections.
40	Poor	Significant UI issues. Many features missing or non-functional. Application crashes or is barely usable. No responsive design consideration. Submission falls well short of minimum frontend requirements.

2. Backend Evaluation

Weight: 20% | Evaluates the Express.js + Prisma backend including API design, database modeling (PostgreSQL), authentication/authorization, middleware usage, error handling, and adherence to REST conventions.

Score	Label	Descriptor
100	Excellent	Well-structured layered architecture (controller/service/repository). All APIs secured with proper JWT auth and RBAC. Consistent JSON error responses. SQL transactions on multi-table operations. No sensitive data exposed. Environment variables used correctly throughout.
90	Very Good	Clean code structure with good separation of concerns. Proper authentication middleware and role-based guards. Global error handler present. Most edge cases handled. Minor architectural inconsistencies.
80	Good	Functional APIs meeting core requirements. JWT auth implemented correctly. Prisma ORM used appropriately. Some missing validation or inconsistent error format. Minor security considerations overlooked.

Score	Label	Descriptor
70	Above Average	Core API endpoints work but some are unsecured. Basic validation present. Error handling exists but inconsistent. Database relations defined but may lack proper constraints or soft delete.
60	Average	Partially meets requirements. Several endpoints missing, broken, or unsecured. Limited validation. Hard deletes used where soft delete is required. Logic mixed in controllers without service layer separation.
40	Poor	Significant backend issues. Most features non-functional or inaccessible via API. No meaningful auth/authorization. No error handling. Credentials possibly hardcoded. Database design is incomplete or incorrect.

3. Core Features

Weight: 20% | Evaluates how completely and correctly the required technical features are implemented: Auth & RBAC, CRUD with relationships, Search/Filter/Pagination, Transactional flow, Dashboard with data viz, File upload (cloud), and Mailer system.

Score	Label	Descriptor
100	Excellent	All 7 core technical requirements fully implemented and working end-to-end in production. Edge cases handled (rollback on failure, expiry logic, async email). Dashboard data aggregated server-side with working charts. File upload with cloud storage and old file cleanup.
90	Very Good	All or nearly all core features functional. Minor gaps in edge case handling (e.g., missing rollback on one flow, or one dashboard metric missing). Everything deployable and demo-able without major issues.
80	Good	Most core features implemented and working. One or two features have minor bugs or are slightly incomplete (e.g., pagination exists but only frontend-side, or email sends but lacks HTML template).
70	Above Average	About half the core features fully working. Some features partially implemented or have noticeable logic gaps. Transactional flow may be missing rollback. Search/filter may be frontend-only.
60	Average	Several core features missing or broken. Only basic CRUD and auth implemented. Dashboard, mailer, or file upload absent. Search/pagination not implemented at API level.
40	Poor	Only the most basic features attempted and most are non-functional. Core business flow cannot be demonstrated. Minimum requirements are largely unmet.

4. Feature Testing

Weight: 10% | Evaluates application stability during live demo: absence of crashes, console errors, unhandled edge cases, and overall reliability. Also includes unit tests if implemented (bonus).

Score	Label	Descriptor
100	Excellent	Zero crashes or critical errors during demo. All tested flows work flawlessly. Graceful handling of invalid input, empty states, and network errors. Unit tests present and passing (bonus).
90	Very Good	No major bugs. Only a few minor UI glitches or edge case warnings that do not impact core functionality. Application fully stable throughout demo.
80	Good	Minor bugs present that don't affect core flow. One or two non-critical features have issues. Application remains usable and recoverable throughout demo.

Score	Label	Descriptor
70	Above Average	Both major and minor bugs present but the app remains usable with workarounds. Core flow still demonstrable despite some interruptions.
60	Average	Major bugs disrupt usability in key features. Some core flows fail during demo. Errors appear in console. Application works partially but requires significant workarounds.
40	Poor	Major bugs severely impact functionality. Core features cannot be demonstrated. Application crashes frequently or key pages fail to load. Demo is largely unsuccessful.

5. Industry Readiness

Weight: 10% | Evaluates the student's adherence to industry standards: code quality (naming, structure, TypeScript types), security practices (.env, no hardcoded secrets), deployment, GitHub workflow (commits, branching), and README documentation.

Score	Label	Descriptor
100	Excellent	Production-grade code quality. All credentials in .env, .gitignore configured correctly. 20+ meaningful commits with conventional commit messages. Feature branching used. README complete with ERD, demo accounts, and deployment URLs. Application fully deployed and accessible.
90	Very Good	Strong adherence to industry standards with only minor gaps. Deployed and accessible. Good commit history. README mostly complete. Minor code quality issues (a few any types, minor unused code).
80	Good	Generally follows good practices. Application deployed. Reasonable commit history. README present. Some code quality issues: inconsistent error handling, unused console.logs, or minor security gaps.
70	Above Average	Moderate adherence to standards. Deployment attempted. Commit history exists but lacks conventional format. README minimal. Several code quality issues: hardcoded values, missing validation, no branching strategy.
60	Average	Limited standards compliance. Possibly not deployed or deployment broken. Few meaningful commits. No README or very incomplete. Multiple code quality issues. Some credentials possibly exposed.
40	Poor	Very limited industry readiness. Application not deployed. No meaningful Git history or all pushed to main. No README. Credentials hardcoded. Code is disorganized and difficult to maintain.

6. Code Understanding

Weight: 10% | Assessed individually during the demo session. Evaluates the student's ability to explain their own code, answer technical questions, demonstrate understanding of architecture decisions, and articulate how each feature works — separately from their group partner.

Score	Label	Descriptor
100	Excellent	Answers all technical questions with depth and clarity. Can explain architectural decisions, tradeoffs, and improvements. Demonstrates strong conceptual understanding beyond just the code written. Confident and precise under follow-up questions.
90	Very Good	Clearly explains all features they worked on. Handles most technical questions well with only minor gaps. Understands how their code fits into the broader system. Confident under examination.

Score	Label	Descriptor
80	Good	Can explain most of their own code. Occasionally needs prompting but recovers well. Understands core concepts like JWT, middleware, Prisma relations. Some answers lack depth on edge cases or design rationale.
70	Above Average	Can explain basic features but struggles with deeper questions. Some code cannot be explained clearly. Understands what the code does at a surface level but not always why design choices were made.
60	Average	Struggles to explain significant portions of the code. Cannot answer basic technical questions about their own implementation. Shows limited understanding of core concepts (e.g., JWT, RBAC, transactions).
40	Poor	Cannot explain the code they supposedly wrote. Unable to identify where key features are implemented. Strong indication of over-reliance on AI tools without understanding. Fails basic conceptual questions.

7. Presentation

Weight: 10% | Assessed individually during the demo session. Evaluates how well the student communicates their project — clarity of delivery, ability to walk through features, structure of the demo, and responsiveness to examiner questions beyond code understanding.

Score	Label	Descriptor
100	Excellent	Speaks clearly and confidently throughout. Demo is well-structured and covers all key features. Actively engages with examiners, gives precise answers, and elaborates where appropriate. Professional delivery with strong command of the room.
90	Very Good	Communicates well and presents ideas clearly. Demo flows smoothly with only minor gaps. Handles examiner questions with confidence. Good eye contact and pacing.
80	Good	Shares ideas clearly enough and gives a decent demo. Contributes to the presentation and responds to questions adequately. Some minor hesitation or incomplete coverage of certain features.
70	Above Average	Can present basic features and explain the project at a surface level. Occasionally struggles to articulate decisions or skips over important demo steps. Answers questions with some prompting.
60	Average	Presentation is unclear or hard to follow. Struggles with communication and demo flow. Limited engagement with examiners. Key features are either not demonstrated or poorly explained.
40	Poor	Has trouble explaining ideas and delivering a coherent demo. Weak presentation skills with little or no preparation evident. Rarely engages with examiners or team. Demo is largely unsuccessful.